

렌트베네핏 사내 프로그램 설계 스펙

1단계 - 차량DB · 견적~계약 입력 · 캘린더/일정

1. 개요

현재 엑셀(견적서, 계약서, 렌트카 매입차량 리스트, 청구서 등)로 운영 중인 업무를 웹 시스템으로 옮기기 위한 1단계 설계 문서입니다. 목표는 견적서 → 계약 → 차량DB로 이어지는 과정에서 같은 정보를 여러 번 입력하지 않도록 하고, 정기점검·검사·렌트만료·청구서 일정을 자동으로 관리하는 것입니다.

2. 시스템 구조 원칙

- 프론트엔드(화면)와 백엔드(API + 데이터베이스)를 분리한다.
- 지금은 반응형 웹(PC + 모바일 브라우저)으로 시작하고, 같은 백엔드 API를 재사용해 나중에 네이티브 앱을 추가할 수 있도록 한다.
- 고객 개인정보·자금 관련 입력은 PC + 관리자 계정에서만 가능하도록 화면 단에서 제한한다.
- 배포는 Vercel(프론트엔드) + Heroku(백엔드 API 서버) + GitHub(코드 저장소) 조합을 재사용한다.

3. 데이터 모델

아래 표는 기존 엑셀 파일의 실제 컬럼을 기준으로 정리한 테이블 구조입니다. SCHEDULE은 기존에 없던 신규 테이블입니다. 실제 데이터베이스는 MongoDB이므로, 7절에서 이 구조를 컬렉션 단위로 다시 정리합니다.

CUSTOMER (고객)

필드명	설명	타입	비고
고객ID	시스템 내부 식별자	PK	자동 생성
개인/법인명	고객 이름 또는 회사명	문자	필수
사업자/주민번호	사업자등록번호 또는 주민번호	문자	필수
대표자명	법인인 경우 대표자명	문자	법인만

필드명	설명	타입	비고
주소	고객 주소	문자	
담당자명 / 연락처	고객측 실무 담당자	문자	
이메일	청구서 발송 주소	문자	필수
은행명 / 계좌 / 예금주	자동이체용 계좌 정보	문자	

VEHICLE (차량)

필드명	설명	타입	비고
차량코드	예: Ray-001 (차종+순번)	PK	자동 채번
차종 / 차량 사양		문자	필수
연식 / 색상 / 유종 / CC		문자·숫자	
차대번호 (VIN)		문자	필수
차량번호	번호판	문자	등록 후 입력
옵션	블랙박스, 선팅 등	문자	
출고지 주소		문자	
딜러사 / 전시장 / 담당영업사원		문자	

QUOTE (견적서)

필드명	설명	타입	비고
견적ID		PK	자동 생성
고객ID	신규 고객이면 견적 저장 시 고객 동시 생성	FK	
차종 / 사양		문자	필수
총 차량가격		숫자	
예상 월 렌트료 (기간별)	12/24/36/48/60개월 비교	숫자	

필드명	설명	타입	비고
작성일		날짜	자동(오늘)
작성자	로그인한 담당자	문자	자동
상태	진행중 / 계약전환 / 보류	선택	기본값: 진행중

CONTRACT (계약)

필드명	설명	타입	비고
계약번호	형식: 연월+순번 (예: 21100001)	PK	자동 채번
차량코드		FK	계약 전환 시 자동 연결
고객ID		FK	견적에서 자동 연결
견적ID	원본 견적 참조	FK	계약 전환 시 자동 연결
계약사	리스사명 또는 현금	문자	
계약일 / 인도 날짜		날짜	필수
렌트 기간(개월)		숫자	필수
렌트 종료일	계약일 + 기간으로 계산	날짜	자동 계산
지점		문자	
책임담당자 / 실무담당자	사내 담당자	문자	
계약 상태	진행중 / 종료 / 중도해지	선택	기본값: 진행중

PRICING (계약금액)

필드명	설명	타입	비고
계약번호		FK	
기본가격 / 할인금액 / 공급가액		숫자	
택송료 / 취득세 / 공채 / 증지대 /		숫자	

필드명	설명	타입	비고
번호판대 / 등록대행료 / 수수료			
보증금 / 선수금 / 인수가		숫자	
월 렌트료		숫자	필수
월 대여료 결제일 / 계산서발행일		날짜	
위약금률 / 연체이율		숫자(%)	

GIFT (사은품, 계약 1건당 여러 개)

필드명	설명	타입	비고
계약번호		FK	
사은품명		문자	
가격		숫자	

SCHEDULE (일정 / 알림) - 신규

필드명	설명	타입	비고
일정ID		PK	자동 생성
유형	정기점검 / 차량검사 / 렌트만료 / 청구서발송	선택	필수
대상	차량코드 또는 계약번호	FK	
예정일	차량·계약 데이터 기준 자동 계산	날짜	자동 계산
상태	예정 / 완료 / 알림발송됨	선택	기본값: 예정
담당자		문자	

4. 화면 설계

1) 견적서 입력 화면

고객 상담 내용을 기록하고, 계약 이전 단계를 관리한다.

- 입력 항목: QUOTE 테이블 전체 + 신규 고객이면 CUSTOMER 기본 정보 동시 입력
- 액션: 저장 / 계약 전환
- 계약 전환 클릭 시 계약 등록 화면으로 이동하며 고객·차종·가격 정보가 자동으로 채워짐

2) 계약 등록 화면

견적을 실제 계약으로 확정하고, 계약 서류 정보를 추가한다.

- 자동 채움: 고객정보, 차종, 예상 가격 (견적서에서 그대로 가져옴)
- 신규 입력: 차대번호, 차량번호, 계약사, 인도날짜, 담당자, 가격 세부항목(PRICING), 사은품
- 저장 시 자동 처리: 차량코드·계약번호 채번, 렌트종료일 계산, SCHEDULE 자동 생성

3) 차량DB 목록 / 상세 화면

전체 차량과 계약 현황을 조회·검색한다.

- 목록 컬럼: 차량코드, 차종, 고객명, 계약상태, 렌트종료일, 남은기간, 담당자
- 필터: 차종, 지점, 담당자, 계약상태, 종료임박순
- 상세 화면: 고객정보 / 가격 / 일정 탭으로 구성해 관련 데이터를 한 화면에서 확인

4) 캘린더 / 일정 화면

구글 캘린더 + 아웃룩 수동 메모로 관리하던 일정을 시스템 안에서 자동으로 보여준다.

- SCHEDULE 데이터를 월간/주간 캘린더로 표시, 유형별 색상 구분
- 일정 클릭 시 관련 계약/차량 상세로 이동
- 임박 일정(D-7, D-1)은 웹/앱 내 알림으로 우선 제공, 카카오톡 연동은 2단계에서 추가

5. 권한 (1단계)

- 1단계는 관리자 3명만 전체 기능(입력·수정·조회)에 접근 가능하도록 한다.
- 일반 직원 10명의 조회 전용 권한 등 세부 권한 체계는 4단계에서 설계한다.

6. 전체 로드맵

단계	범위	비고
1단계	차량DB + 견적~계약 입력 + 캘린더/일정 알림(웹 내)	이번 문서 범위
2단계	청구서 자동발송 + 정산 + 카카오톡 알림톡 연동	
3단계	계약서 / 등록증 / 취득세 등 서류 업로드·관리	
4단계	대시보드 + 관리자/일반 권한 체계	

7. MongoDB 스키마 (Mongoose)

데이터베이스가 MongoDB이므로 3절의 테이블 구조를 컬렉션 단위로 조정했습니다. 계약과 항상 같이 조회되는 가격(PRICING)·사은품(GIFT)은 CONTRACT 문서 안에 포함시키고, 고객·차량·견적·일정은 별도 컬렉션으로 두어 ID로 참조합니다. 아래 코드는 그대로 파일로 저장해 붙여넣어 쓸 수 있는 수준으로 작성했습니다.

공통

```
const mongoose = require('mongoose');
const { Schema } = mongoose;
```

models/Custom.js

```
const CustomerSchema = new Schema({
  name: { type: String, required: true }, // 개인/법인명
  bizNo: { type: String, required: true }, // 사업자/주민번호
  ceoName: String, // 대표자명 (법인)
  address: String,
  contactName: String, // 담당자명
  contactPhone: String, // 담당자 연락처
  email: { type: String, required: true },
  bank: {
    name: String,
    account: String,
    holder: String, // 예금주
  },
}, { timestamps: true });

module.exports = mongoose.model('Customer', CustomerSchema);
```

models/Vehicle.js

```
const VehicleSchema = new Schema({
  code: { type: String, required: true, unique: true }, // 차량코드 예: Ray-001
```

```

model: { type: String, required: true },    // 차종
spec: String,                             // 차량 사양
year: Number,
color: String,
fuelType: String,
cc: Number,
vin: { type: String, required: true },      // 차대번호
plateNo: String,                          // 차량번호
options: [String],                        // 블랙박스, 선팅 등
releaseAddress: String,                  // 출고지 주소
dealer: String,
salesRep: String,
showroom: String,
}, { timestamps: true });

module.exports = mongoose.model('Vehicle', VehicleSchema);

```

models/Quote.js

```

const QuoteSchema = new Schema({
  customer: { type: Schema.Types.ObjectId, ref: 'Customer', required: true },
  vehicleModel: String,
  vehicleSpec: String,
  totalPrice: Number,
  monthlyEstimates: [{ termMonths: Number, monthlyFee: Number }],
  status: { type: String, enum: ['진행중', '계약전환', '보류'], default: '진행중' },
  createdBy: String,
}, { timestamps: true });

module.exports = mongoose.model('Quote', QuoteSchema);

```

models/Contract.js

가격(pricing)과 사은품(gifts)을 하위 문서로 포함하고, 저장 시 렌트종료일을 자동 계산합니다.

```

const ContractSchema = new Schema({
  contractNo: { type: String, required: true, unique: true }, // 자동 채번
  vehicle: { type: Schema.Types.ObjectId, ref: 'Vehicle', required: true },
  customer: { type: Schema.Types.ObjectId, ref: 'Customer', required: true },
  quote: { type: Schema.Types.ObjectId, ref: 'Quote' },
  leaseCompany: String, // 계약사
  contractDate: Date,
  deliveryDate: Date,
  termMonths: Number,
  endDate: Date, // pre-save 혹에서 자동 계산
  branch: String,
  managerMain: String, // 책임담당자
  managerOps: String, // 실무담당자
  status: { type: String, enum: ['진행중', '종료', '중도해지'], default: '진행중' },

  pricing: {
    basePrice: Number,
    discount: Number,

```

```

    supplyPrice: Number,
    deliveryFee: Number,
    acquisitionTax: Number,
    publicBond: Number,
    stampFee: Number,
    plateFee: Number,
    registrationAgencyFee: Number,
    commission: Number,
    deposit: Number,
    advancePayment: Number,
    takeoverPrice: Number,
    monthlyFee: { type: Number, required: true },
    billingDay: Number,
    invoiceDay: Number,
    penaltyRate: Number,
    overdueRate: Number,
  },

  gifts: [{ name: String, price: Number } ],
}, { timestamps: true });

ContractSchema.pre('save', function (next) {
  if (this.contractDate && this.termMonths) {
    const end = new Date(this.contractDate);
    end.setMonth(end.getMonth() + this.termMonths);
    this.endDate = end;
  }
  next();
});

module.exports = mongoose.model('Contract', ContractSchema);

```

models/Schedule.js

정기점검·차량검사·렌트만료·청구서발송 일정을 저장하는 컬렉션입니다. dueDate에 인덱스를 걸어 캘린더 화면에서 특정 기간의 일정을 빠르게 조회할 수 있게 했습니다.

```

const ScheduleSchema = new Schema({
  type: {
    type: String,
    enum: ['정기점검', '차량검사', '렌트만료', '청구서발송'],
    required: true,
  },
  targetVehicle: { type: Schema.Types.ObjectId, ref: 'Vehicle' },
  targetContract: { type: Schema.Types.ObjectId, ref: 'Contract' },
  dueDate: { type: Date, required: true, index: true },
  status: { type: String, enum: ['예정', '완료', '알림발송됨'], default: '예정' },
  assignee: String,
}, { timestamps: true });

module.exports = mongoose.model('Schedule', ScheduleSchema);

```